

# Apache Kafka

Java Backend Interview — Short Notes

## 1. Core Concepts

| Concept               | Description                                                          | Key Points                                                                       |
|-----------------------|----------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>Broker</b>         | Kafka server — stores messages, serves producers/consumers           | Cluster = multiple brokers. Leader per partition. JVM process                    |
| <b>Topic</b>          | Named stream of records — logical channel for messages               | Partitioned for parallelism. Replicated for durability                           |
| <b>Partition</b>      | Ordered, immutable sequence of records within a topic                | Unit of parallelism. More partitions = more consumer parallelism                 |
| <b>Offset</b>         | Unique sequential ID of record within partition                      | Consumer tracks offset — controls what's been consumed                           |
| <b>Producer</b>       | Writes records to topics                                             | Chooses partition by key (consistent hash) or round-robin                        |
| <b>Consumer</b>       | Reads records from topics by polling                                 | Tracks own offset in <code>__consumer_offsets</code> topic                       |
| <b>Consumer Group</b> | Group of consumers sharing partition consumption                     | Each partition → exactly one consumer in group. Max consumers = partitions       |
| <b>Leader</b>         | Partition replica that handles all reads/writes                      | Each partition has 1 leader. Other replicas are followers                        |
| <b>Follower/ISR</b>   | Replica that replicates leader. ISR = In-Sync Replica                | ISR set: replicas caught up with leader. <code>acks=all</code> writes to all ISR |
| <b>ZooKeeper</b>      | Coordination service — cluster metadata, leader election (pre-KRaft) | Kafka 3.3+ KRaft mode removes ZooKeeper dependency                               |
| <b>KRaft</b>          | Kafka Raft — built-in metadata management replacing ZooKeeper        | Simpler ops, faster recovery. Production-ready in Kafka 3.3+                     |
| <b>Retention</b>      | How long Kafka keeps messages — time or size based                   | Default 7 days. Consumers can replay any time within retention                   |
| <b>Compaction</b>     | Keep only latest value per key — useful for event sourcing           | <code>log.cleanup.policy=compact</code> . Deletes old records with same key      |

## 2. Producer Deep Dive

### 2.1 Producer Acknowledgments (acks)

| acks            | Durability                                                              | Performance            | Use When                                                                |
|-----------------|-------------------------------------------------------------------------|------------------------|-------------------------------------------------------------------------|
| <b>0</b>        | No ack — fire and forget                                                | Fastest — no wait      | Metrics, logs — loss acceptable                                         |
| <b>1</b>        | Leader ack only — data lost if leader fails before follower replicates  | Fast                   | Good balance — most use cases                                           |
| <b>all (-1)</b> | All ISR must ack — no data loss if <code>min.insync.replicas</code> met | Slowest — most durable | Financial, critical events. Use with <code>min.insync.replicas=2</code> |

## 2.2 Key Producer Configs

| Config                        | Description                                                          | Recommended                                                     |
|-------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>acks</b>                   | Acknowledgment level (0, 1, all)                                     | all for critical data                                           |
| <b>retries</b>                | Number of retries on transient failure                               | Integer.MAX_VALUE with idempotent=true                          |
| <b>enable.idempotence</b>     | Exactly-once delivery to broker — dedup by producer ID + sequence    | true — no duplicate messages on retry                           |
| <b>batch.size</b>             | Max bytes per batch sent to partition                                | Default 16KB. Increase for throughput: 32-64KB                  |
| <b>linger.ms</b>              | Wait up to N ms to fill batch — trade latency for throughput         | 0 (low latency) to 20 (higher throughput)                       |
| <b>compression.type</b>       | Compress batches: none, gzip, snappy, lz4, zstd                      | lz4 (fast) or zstd (best ratio) for production                  |
| <b>max.in.flight.requests</b> | Unacked requests per connection                                      | 1 for strict ordering (without idempotence). 5 with idempotence |
| <b>buffer.memory</b>          | Total bytes producer can buffer                                      | Default 32MB. Increase for high-throughput producers            |
| <b>transactional.id</b>       | Enable transactions — exactly-once across multiple partitions/topics | Required for transactional producers                            |

## 2.3 Partitioning Strategy

- With key:  $\text{hash}(\text{key}) \% \text{numPartitions}$  — same key always same partition — ordering guaranteed per key
- Without key: sticky partitioner (Kafka 2.4+) — fill batch then move to next partition
- Custom: implement Partitioner interface — route by geography, tenant, load

```
// Spring Kafka producer
@Autowired KafkaTemplate kafkaTemplate;
// Send with key – same orderId always same partition
kafkaTemplate.send("order.created", order.getId().toString(), event);
// Or fluent API
kafkaTemplate.send(new ProducerRecord<>("order.created", partition, key, event));
```

## 3. Consumer Deep Dive

### 3.1 Consumer Group Mechanics

- Partitions distributed across consumers in group. K partitions, N consumers:
  - $N < K$ : some consumers get multiple partitions
  - $N = K$ : each consumer gets exactly one partition — ideal
  - $N > K$ : extra consumers idle — no work to do
- **Rebalance**: triggered when consumer joins/leaves/crashes. Partitions reassigned. No consumption during rebalance
- **Cooperative rebalancing** (Kafka 2.4+): incremental — only moved partitions paused. Reduces rebalance impact

### 3.2 Offset Management

- Offsets stored in `__consumer_offsets` topic. Committed by consumer after processing
- **auto.offset.reset**: what to do when no committed offset. latest (skip old), earliest (from beginning), none (error)
- **enable.auto.commit=true** (default): commits every `auto.commit.interval.ms`. Risk: commit before processing = at-most-once

- **enable.auto.commit=false**: manual commit. `commitSync()` — blocks. `commitAsync()` — non-blocking. Enables at-least-once

```
// Spring Kafka – manual ack (at-least-once)
@KafkaListener(topics="order.created", groupId="payment-svc")
public void handle(OrderEvent event, Acknowledgment ack) {
    try {
        paymentService.process(event);
        ack.acknowledge(); // commit offset only after successful processing
    } catch (RetryableException e) {
        // don't ack – will be redelivered
    } catch (FatalException e) {
        ack.acknowledge(); // ack to skip – send to DLT manually
    }
}
```

### 3.3 Key Consumer Configs

| Config                       | Description                                                   | Notes                                             |
|------------------------------|---------------------------------------------------------------|---------------------------------------------------|
| <b>group.id</b>              | Consumer group identifier                                     | All consumers with same group.id share partitions |
| <b>auto.offset.reset</b>     | earliest / latest / none                                      | earliest: replay all. latest: only new messages   |
| <b>enable.auto.commit</b>    | Auto-commit offsets periodically                              | false for at-least-once. Manual ack required      |
| <b>max.poll.records</b>      | Max records returned per poll()                               | Default 500. Reduce for slow processing consumers |
| <b>max.poll.interval.ms</b>  | Max time between poll() calls before consumer considered dead | Default 5min. Increase for slow batch processing  |
| <b>session.timeout.ms</b>    | Heartbeat timeout — consumer considered dead                  | Default 10s. Lower = faster failure detection     |
| <b>heartbeat.interval.ms</b> | How often consumer sends heartbeat                            | Default 3s. Must be < session.timeout.ms / 3      |
| <b>fetch.min.bytes</b>       | Min data broker returns per fetch                             | Increase for throughput: 1MB                      |
| <b>fetch.max.wait.ms</b>     | Max wait if insufficient data                                 | Default 500ms                                     |

## 4. Delivery Guarantees

| Guarantee            | Producer                               | Consumer                       | Risk               | Use Case                                  |
|----------------------|----------------------------------------|--------------------------------|--------------------|-------------------------------------------|
| <b>At-most-once</b>  | acks=0 or 1, no retry                  | Commit before process          | Message loss       | Metrics, non-critical logs                |
| <b>At-least-once</b> | acks=all, retries enabled              | Commit after process           | Duplicate delivery | Most business events — handle idempotency |
| <b>Exactly-once</b>  | enable.idempotence=true + transactions | Isolation level read_committed | Highest overhead   | Financial transactions, balance updates   |

- **Exactly-once within Kafka** — idempotent producer (no duplicate on retry) + transactions (atomic write to multiple partitions/topics)

- **End-to-end exactly-once** — Kafka EOS for read-process-write within Kafka. For external systems (DB), need idempotent consumer + transactional outbox

```
// Transactional producer
@Bean public ProducerFactory producerFactory() {
    Map config = new HashMap<>();
    config.put(TRANSACTIONAL_ID_CONFIG, "tx-1");
    config.put(ENABLE_IDEMPOTENCE_CONFIG, true);
    return new DefaultKafkaProducerFactory<>(config);
}
// Spring KafkaTransactionManager for @Transactional integration
```

## 5. Kafka Internals

### 5.1 Log Storage

- Each partition = append-only log files on disk. Segments: .log (data) + .index (offset→position) + .timeindex
- Sequential writes — OS page cache + sequential I/O = very fast (≈ disk write speed)
- Zero-copy transfer (sendfile syscall) — data goes kernel buffer → NIC directly, no user space copy
- Consumers can read from page cache when up-to-date — no disk access needed

### 5.2 Replication

- Each partition has 1 leader + (replication.factor-1) followers
- ISR (In-Sync Replicas): replicas within replica.lag.time.max.ms of leader
- **min.insync.replicas**: minimum ISR required for acks=all to succeed. Default 1 — set to 2 for durability
- If leader fails: controller elects new leader from ISR. Failover in seconds

### 5.3 Consumer Lag & Monitoring

- **Consumer lag**: difference between latest offset in partition and consumer's committed offset
- High lag = consumer falling behind. Alert when lag grows continuously
- Monitor: kafka-consumer-groups.sh --describe, Burrow, Prometheus kafka\_consumer\_group\_lag metric
- Auto-scaling: HPA on Kafka consumer lag metric (KEDA — Kubernetes Event-Driven Autoscaler)

## 6. Spring Kafka

### 6.1 Configuration

```
spring:
  kafka:
    bootstrap-servers: kafka1:9092,kafka2:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: all
    properties:
      enable.idempotence: true
    consumer:
      group-id: order-service
      auto-offset-reset: earliest
      enable-auto-commit: false
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
```

```

value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
properties:
spring.json.trusted.packages: com.example.events
listener:
ack-mode: MANUAL_IMMEDIATE
concurrency: 3 # 3 listener threads per @KafkaListener

```

## 6.2 @KafkaListener Advanced

```

@KafkaListener(
    topics = "order.created",
    groupId = "payment-svc",
    concurrency = "3",
    containerFactory = "kafkaListenerContainerFactory")
public void handle(@Payload OrderEvent event,
    @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
    @Header(KafkaHeaders.OFFSET) long offset,
    Acknowledgment ack) {
    log.info("Partition: {}, Offset: {}", partition, offset);
    paymentService.process(event);
    ack.acknowledge();
}

```

## 6.3 Dead Letter Topic (DLT)

- Failed messages after max retries sent to DLT — topic.DLT suffix
- Spring Kafka RetryableTopic: @RetryableTopic annotation handles retry + DLT automatically

```

@RetryableTopic(
    attempts = "4", // 1 original + 3 retries
    backoff = @Backoff(delay=1000, multiplier=2.0), // exp backoff
    dltTopicSuffix = ".DLT",
    include = {RetryableException.class})
@KafkaListener(topics="order.created")
public void handle(OrderEvent event) { ... }

@DltHandler
public void handleDlt(OrderEvent event, @Header(KafkaHeaders.RECEIVED_TOPIC) String topic) {
    log.error("DLT received from {}: {}", topic, event);
    alertingService.sendAlert(event);
}

```

## 7. Kafka Patterns

### 7.1 Kafka vs RabbitMQ vs SQS

|                   | Kafka                              | RabbitMQ                   | AWS SQS                                      |
|-------------------|------------------------------------|----------------------------|----------------------------------------------|
| <b>Model</b>      | Pull-based. Consumer group         | Push-based. AMQP           | Pull-based. Managed queue                    |
| <b>Retention</b>  | Configurable (days/weeks). Replay  | Until consumed (no replay) | Up to 14 days. No replay                     |
| <b>Throughput</b> | Millions msg/sec — partitioned log | Thousands msg/sec          | Thousands msg/sec (standard), ~3000/s (FIFO) |
| <b>Ordering</b>   | Per partition (guaranteed by key)  | Per queue                  | FIFO queue — per group                       |

|                        |                                                         |                                 |                                                    |
|------------------------|---------------------------------------------------------|---------------------------------|----------------------------------------------------|
| <b>Message size</b>    | Default 1MB (configurable)                              | Default 128MB                   | 256KB (S3 extended = 2GB)                          |
| <b>Consumer groups</b> | Multiple independent groups — full replay each          | Competing consumers on queue    | Multiple consumers — message deleted after consume |
| <b>Use for</b>         | Event streaming, audit log, event sourcing, high-volume | Task queues, routing, RPC-style | AWS-native task queues, SQS+SNS fan-out            |

## 7.2 Key Patterns

- **Event Streaming**: real-time pipelines — user activity → Kafka → analytics, personalization
- **Change Data Capture (CDC)**: Debezium reads DB transaction log → Kafka. Zero-latency DB change events
- **CQRS + Event Sourcing**: write events to Kafka → multiple consumers build different read models
- **Saga Choreography**: services react to events on Kafka topics — distributed transaction coordination
- **Outbox Pattern**: save entity + event in same DB transaction → CDC/relay publishes to Kafka
- **Compacted Topic**: latest state per key — effectively a distributed key-value store (like Redis)

## 8. Interview Questions

### Core Architecture

#### Q1. What is Kafka? How does it differ from traditional message queues?

→ Kafka: distributed event streaming platform — append-only log. Traditional MQ (RabbitMQ): message deleted after consumed, push-based. Kafka: message retained (days/weeks), pull-based, replay supported, multiple independent consumer groups, massive throughput via partitioning

#### Q2. What is a partition and why is it important?

→ Partition: ordered immutable sequence within topic. Enables parallelism — N partitions = N consumers in parallel. Ordering guaranteed within partition. More partitions = higher throughput but more overhead. Rule: partitions >= consumers in group

#### Q3. What is a consumer group?

→ Group of consumers sharing work across topic partitions. Each partition assigned to exactly one consumer in group. Add consumers to scale up to limit of partition count. Multiple groups = multiple independent consumers of same topic (each group gets all messages)

#### Q4. How does Kafka guarantee ordering?

→ Within a partition: strictly ordered by offset. Across partitions: no ordering. Use key-based partitioning (same key → same partition) for ordering per entity. For global ordering: use 1 partition (limits throughput) or business-level sequencing

#### Q5. What is ISR? What is min.insync.replicas?

→ ISR: In-Sync Replicas — replicas caught up with leader. With acks=all, leader waits for all ISR to acknowledge before responding. min.insync.replicas=2 means at least 2 replicas must be in sync — prevents data loss if leader fails

### Producers & Consumers

#### Q6. Explain acks=all vs acks=1 vs acks=0?

→ acks=0: no wait — fastest, loss possible. acks=1: leader ack — medium durability, loss if leader fails before replication. acks=all: all ISR ack — strongest durability, slowest. Use acks=all + min.insync.replicas=2 for no data loss

#### Q7. What is idempotent producer?

→ enable.idempotence=true: producer assigned unique ID + sequence number. Broker deduplicates retries — no duplicate messages. Enables exactly-once within Kafka. Required for transactional producers. Automatically sets acks=all, max.in.flight=5

#### Q8. At-most-once vs at-least-once vs exactly-once?

→ At-most-once: commit before process — may miss. At-least-once: commit after process — may duplicate. Exactly-once: idempotent producer + transactions (producer) + read\_committed isolation (consumer). External systems need idempotent consumer logic

#### Q9. What is consumer lag? Why is it important?

→ Lag = latest offset - consumer committed offset per partition. Indicates how far behind consumer is. Growing lag = consumer slower than producer. Alert on growing lag. Scale consumers (add instances up to partition count) or optimize processing

#### **Q10. What is rebalance? How to minimize its impact?**

→ Rebalance: redistribute partitions when consumer joins/leaves/crashes. All consumers pause during rebalance (eager). Minimize: cooperative rebalance (incremental, only moved partitions pause), static group membership (session.timeout not rebalance trigger), keep consumers healthy

### **Advanced**

#### **Q11. What is Kafka Streams?**

→ Client library for stream processing on Kafka. Stateful and stateless transformations, joins, windowing, aggregations. Runs in your application — no separate cluster. KTable (changelog), KStream (record stream), GlobalKTable

#### **Q12. What is Schema Registry?**

→ Centralized schema store (Avro/Protobuf/JSON schemas). Producers register schema → get schema ID. Consumers fetch schema by ID to deserialize. Enforces compatibility rules (backward, forward, full). Prevents breaking changes breaking consumers

#### **Q13. How do you handle poison pill messages?**

→ Message that always fails processing — blocks partition indefinitely. Fix: DLT (Dead Letter Topic) after max retries. Spring @RetryableTopic handles retries + DLT automatically. Alert on DLT messages, investigate and replay or discard

#### **Q14. What is Kafka compaction?**

→ log.cleanup.policy=compact: Kafka keeps only the latest record per key. Old records with same key deleted. Use for: state store (latest user state), event sourcing snapshots, distributed cache. tombstone (null value) = delete key

#### **Q15. Kafka transactions — how do they work?**

→ transactional.id enables atomic writes across multiple partitions/topics. Producer begins transaction, writes to multiple topics, commits atomically. Consumer with isolation.level=read\_committed only sees committed data. Enables exactly-once read-process-write within Kafka

### **Spring Kafka**

#### **Q16. @KafkaListener concurrency setting?**

→ concurrency=3 creates 3 listener threads — each consuming different partition assignments. Effective parallelism = min(concurrency, partitions). More threads than partitions = idle threads

#### **Q17. How does Spring Kafka RetryableTopic work?**

→ @RetryableTopic: on failure, message sent to retry topic (topic-retry-0, topic-retry-1...) with delay. After max attempts, goes to DLT. Retry topics are separate Kafka topics — original partition not blocked. @DltHandler processes DLT messages

#### **Q18. How do you test Kafka consumers in Spring Boot?**

→ Use @EmbeddedKafka — in-memory Kafka broker for tests. KafkaTemplate to send test messages. @KafkaListener receives them. CountDownLatch or Awaitility to wait for async processing. Alternatively use Testcontainers with real Kafka

#### **Q19. How to ensure message ordering in Spring Kafka?**

→ Use message key — same key → same partition → ordered. Set concurrency=1 for single-threaded processing. Or per-partition concurrency: each partition processed by one thread. Don't use async processing inside listener if ordering matters

#### **Q20. What is KafkaTemplate sendResult?**

→ ListenableFuture (or CompletableFuture) returned by send(). Use .get() for synchronous confirmation. Or addCallback for async. ProducerRecord with key ensures partition. Use send(ProducerRecord) for full control over key/partition/headers/timestamp